



Antes de empezar...

- Datos curiosos
 - 40 personas inscritas
 - 45 personas en classroom
- Dudas tarea 1
- Los grupos para asesorías

Los grupos

Teresa Beceril (tbt2415@gmail.com)

ABREU LLANES ANABEL
ÁLVAREZ PÉREZ PABLO ALBERTO
ÁVILA GARCÍA DAVID
BELMONT HERNANDEZ HECTOR ANTONIO
CAMPOS REYES MARCO ANTONIO
CARRILLO CASTAÑEDA SANTIAGO AUGUSTO KAM
DEL RÍO PULIDO ERIK FEDERICO
FLORES GUAPO IVAN
GARCÍA TORRES MÓNICA
GOMEZ BURBANO ANDRES DAVID

Abril Reyes (abril.reyesflo@gmail.com)

GONZÁLEZ BELLO DANIEL
GUTIÉRREZ GONZÁLEZ CARLOS ARIEL
HERNÁNDEZ ALVAREZ CARLOS ALFREDO
HERNANDEZ ANAYA PEDRO ISAI
HERNÁNDEZ FLORES KARINA
HERNÁNDEZ GONZÁLEZ LUIS ÁNGEL
HERNÁNDEZ GUERRERO OSCAR
HERNÁNDEZ PIEDRAS LIZETH EVELYN
HUITZILT RODRÍGUEZ ARMANDO
IBÁÑEZ CASTRO MARIO IGNACIO

Jorge Calvillo (jorge.calvillo@comunidad.unam.mx)

JIMÉNEZ TERRAZAS LUZ CITLALLI
MALVÁEZ FLORES AXEL DANIEL
MARTÍNEZ HERRERA MIGUEL AGUSTÍN
MARTÍNEZ MANZO LUIS FERMIN
MIRAMONTES SARABIA LINA SOFÍA
NUÑEZ LACES YENI YUZELMI
ORTEGA MEDINA MARCOS
PABLO MARTÍNEZ YESSICA JANETH
PASTRANA CALDERÓN EDGAR
PERALTA CORREA JOSÉ ROBERTO

Alberto Alonso (j.albertoalonso.96@gmail.com)

REBOLLAR HERNÁNDEZ EDGARDO GIOVANNI
RIVERA CAMACHO AMÉRICA ANDREA
RODRÍGUEZ CAMPOS ERICK EDUARDO
RODRÍGUEZ GARCÍA JAVIER ANTONIO
SANTILLAN RODRÍGUEZ ABEL
VARGAS PEREZ EDUARDO
VEGA VILCHIS RODRIGO
VILLA AGUILAR GERARDO ERICK
VILLALBA GONZÁLEZ DIEGO ANTONIO



Notas

- Complejidad BFS
 - En grafos
 - $O(V + E)$. V es el número de vértices y E el número de aristas.
 - En árboles
 - $O(b^m)$. b es el factor de ramificación y m la profundidad del árbol.
 - En algunos lugares verán $O(n)$ ó $O(V)$. n o V es el número de nodos.
 - Nótese que en árboles $E = V - 1$
 - $V + E = V + V - 1 = 2V - 1 = O(V)$
 - $n = V \leq b^m$
- Dijkstra vs Búsqueda de costo uniforme

Inteligencia Artificial

Clase 4: Búsqueda con información



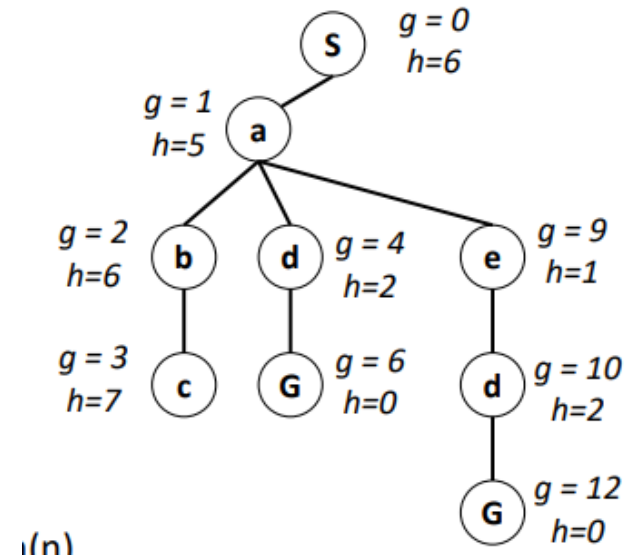
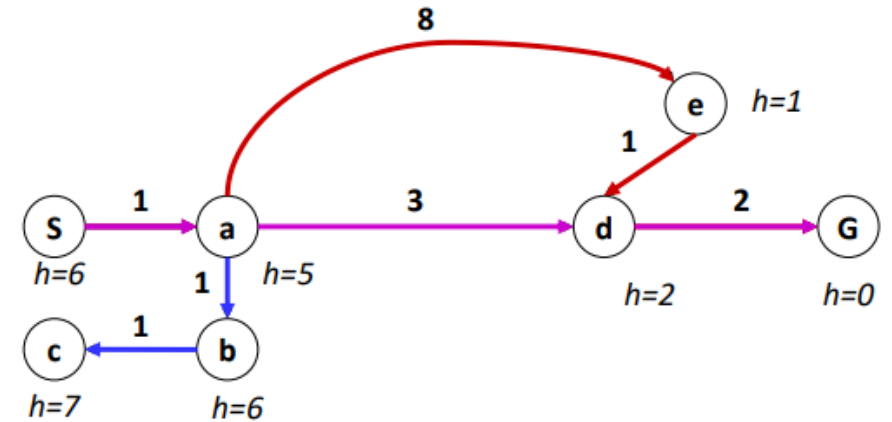
Para el día de hoy

- Búsqueda...
 - A*



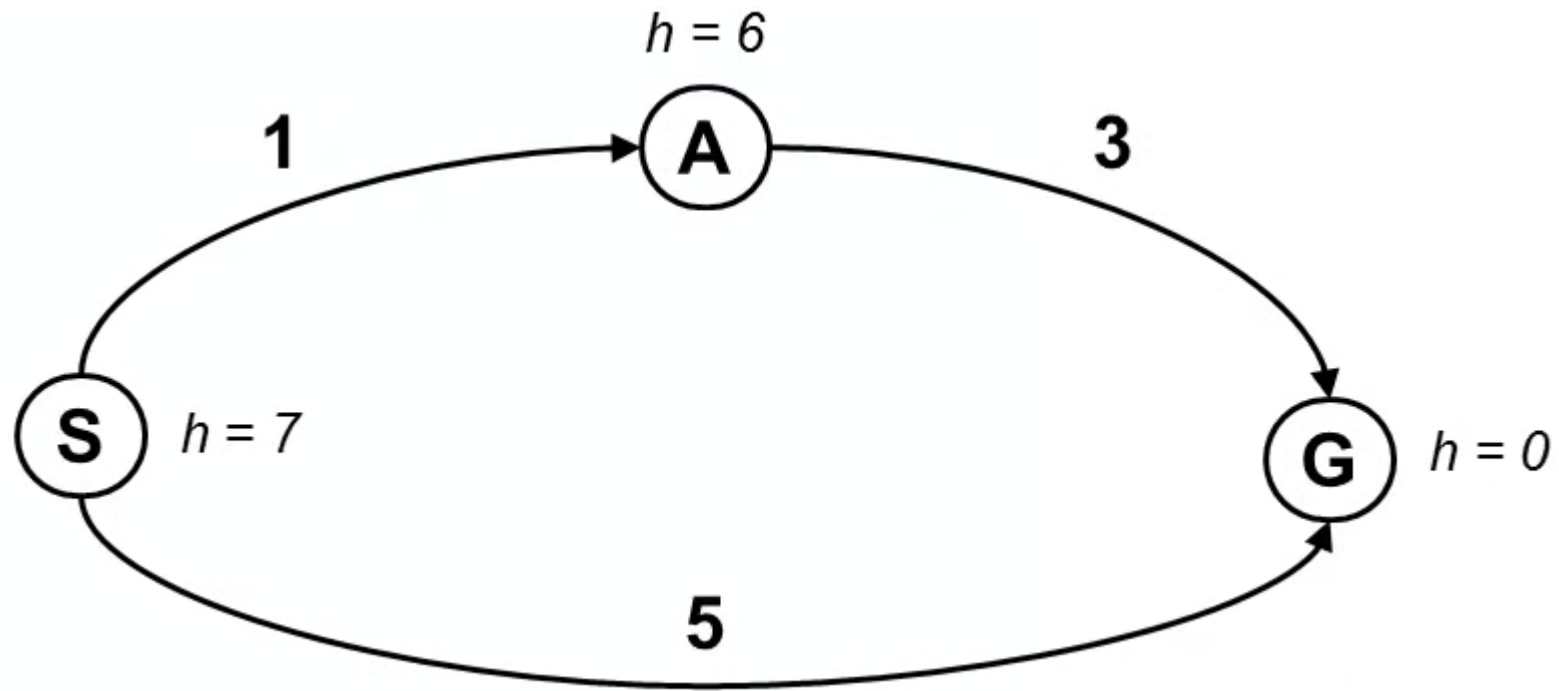
$A^* = \text{Costo uniforme} + \text{voraz}$

- Ordena la búsqueda de acuerdo con la suma de $f(n) = g(n) + h(n)$
- Donde:
 - $g(n)$ es el costo utilizado por costo uniforme
 - $h(n)$ es el costo utilizado por la búsqueda voraz



¿Es A^* óptimo?

Otro ejemplo

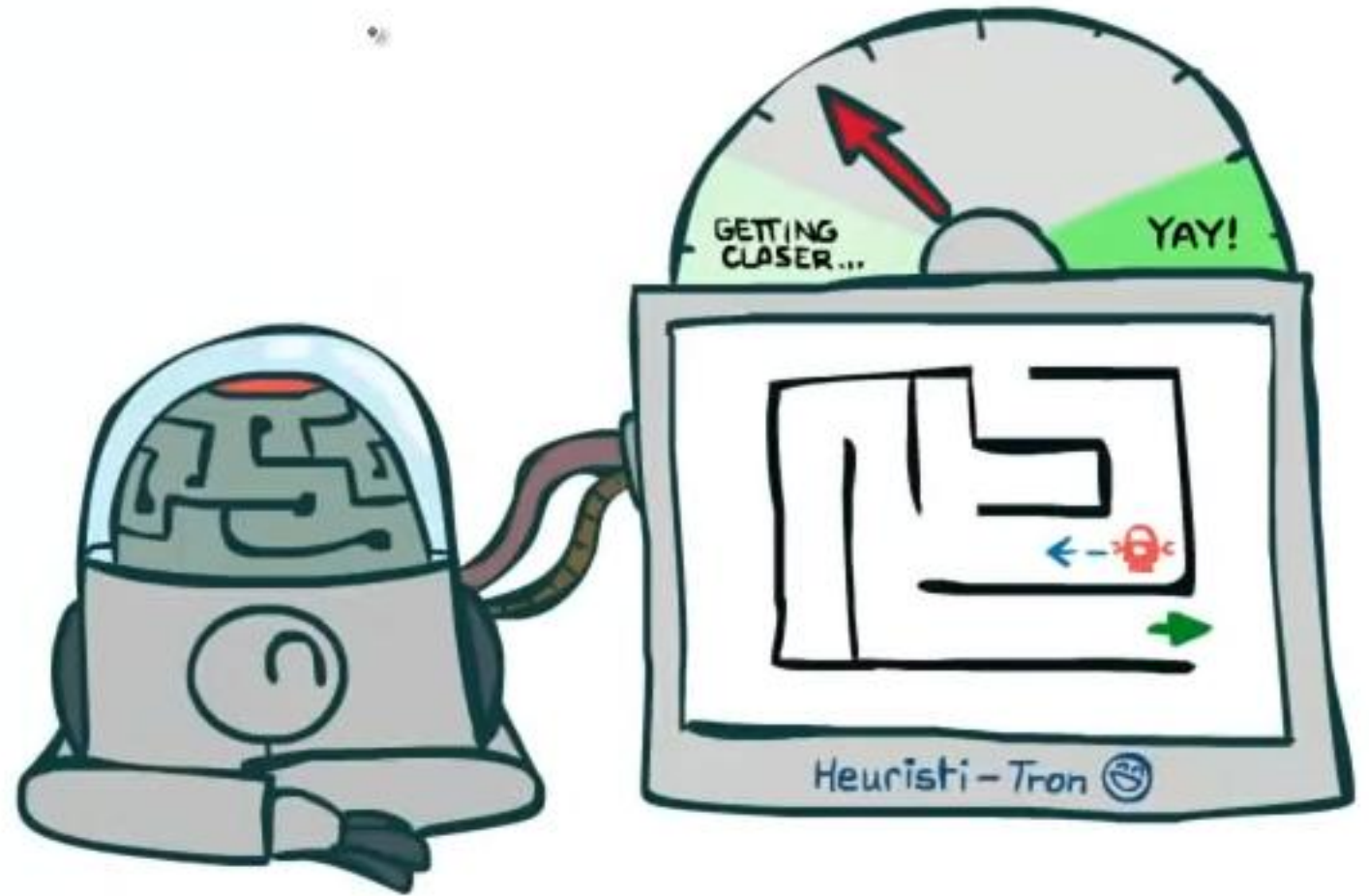


Heurísticas admisibles

- Una heurística h es admisible si

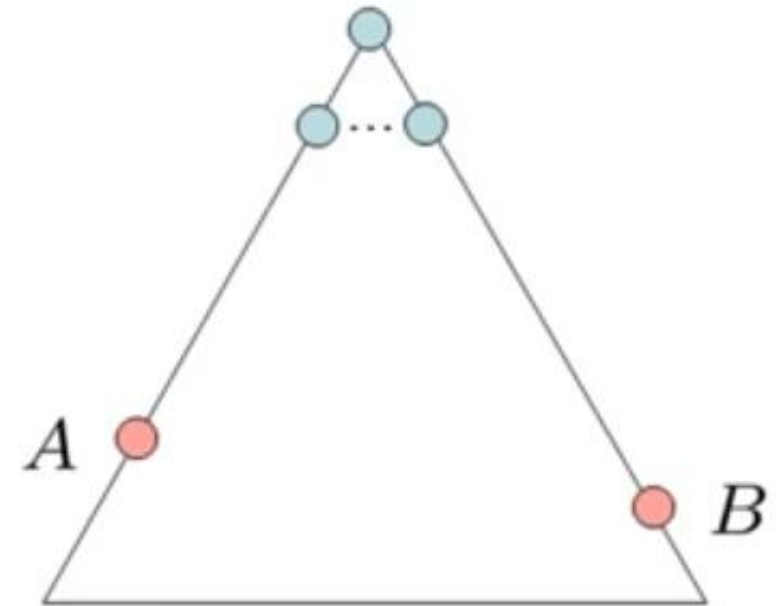
$$0 \leq h(n) \leq h^*(n)$$

Donde $h^*(n)$ es el costo verdadero a la meta más cercana

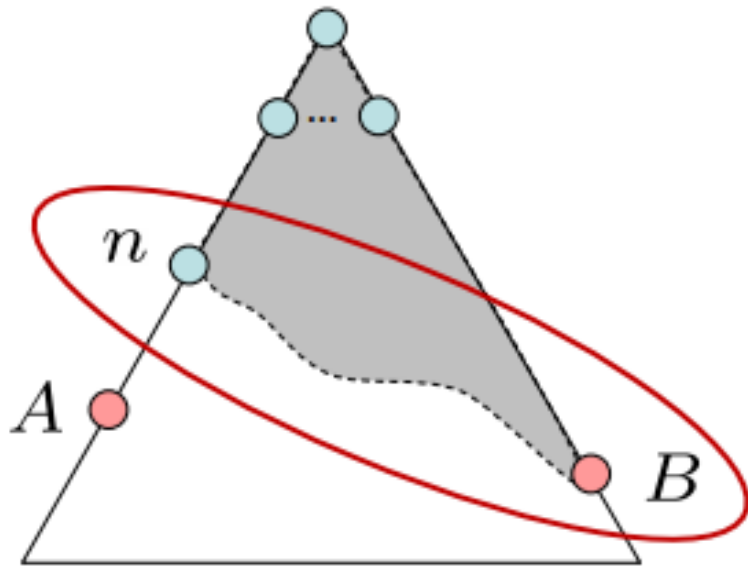


Optimalidad de A^*

- Supongamos que
 - A es un nodo meta óptimo
 - B es un nodo meta suboptimo
 - H es admisible
- Entonces
 - A saldrá de la frontera antes que B



Optimalidad de A^*



- Demostración
 - Supongamos que B está en la frontera
 - Algún ancestro n de A está en la frontera (o A)
 - Recordar que $f(x) = g(x) + h(x)$
- Entonces: n saldrá antes que B
 - $f(n) \leq f(A)$
 - $f(A) < f(B)$
 - n se expande antes que B $f(n) \leq f(A) < f(B)$
- Todos los ancestros de A se expanden antes que B
- A se expande antes que B
- A^* es óptimo

Creando heurísticas admisibles



Es normalmente el punto más importante en el diseño de una heurística



Normalmente son soluciones de un problema relajado



Una heurística inadmisibile puede ayudar en ocasiones

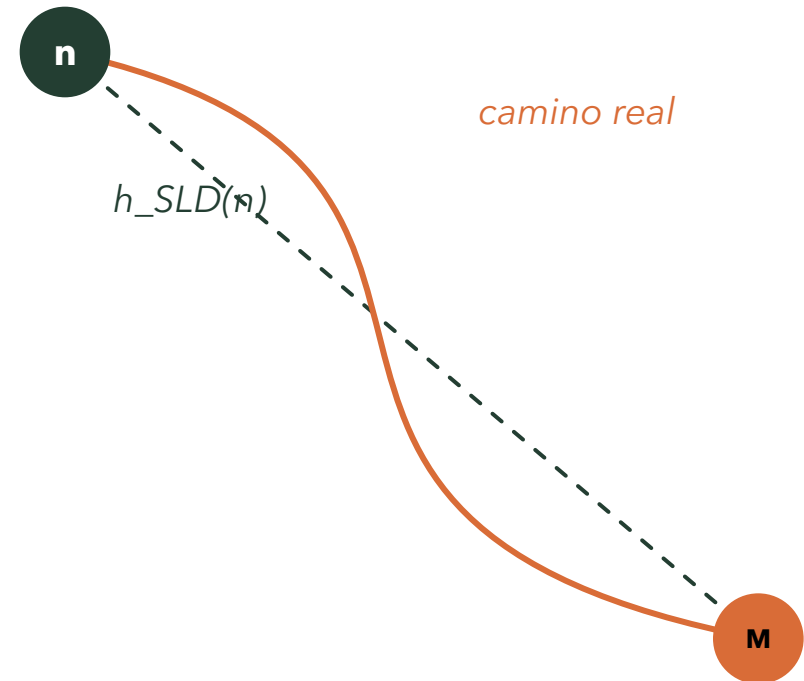
Admisibilidad: Distancia en línea recta

Problema: encontrar la ruta más corta entre dos ciudades en un mapa de carreteras.

Heurística: distancia en línea recta $h_SLD(n)$ = distancia euclidiana de n a la meta.

Proposición: h_SLD es admisible.

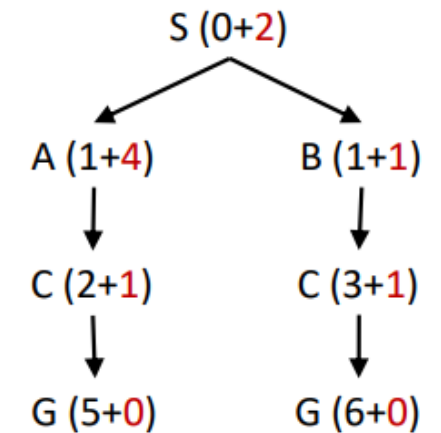
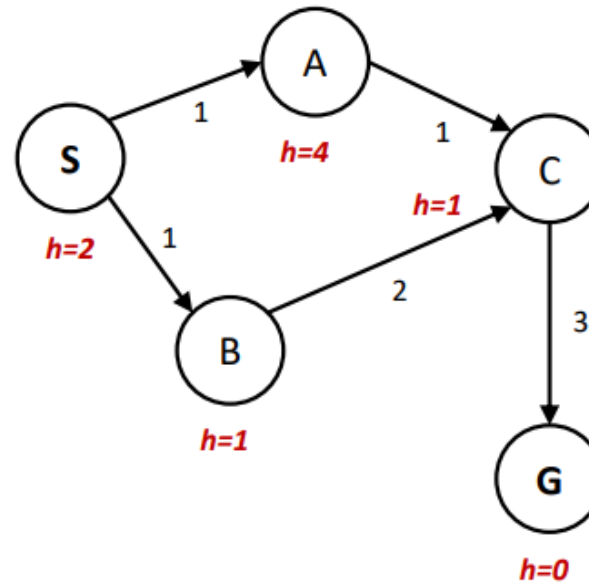
- Prueba: un camino en carretera entre dos puntos debe recorrer al menos la distancia euclidiana,
- ya que la línea recta es el camino más corto en el espacio euclideo.
- Por lo tanto $h_SLD(n) \leq h^*(n)$ para todo nodo n , y $h_SLD(n) \geq 0$.
- *Problema relajado: si pudiéramos volar directamente ignorando carreteras, el costo óptimo sería exactamente $h_SLD(n)$. ■*



$$h_SLD(n) \leq h^*(n)$$

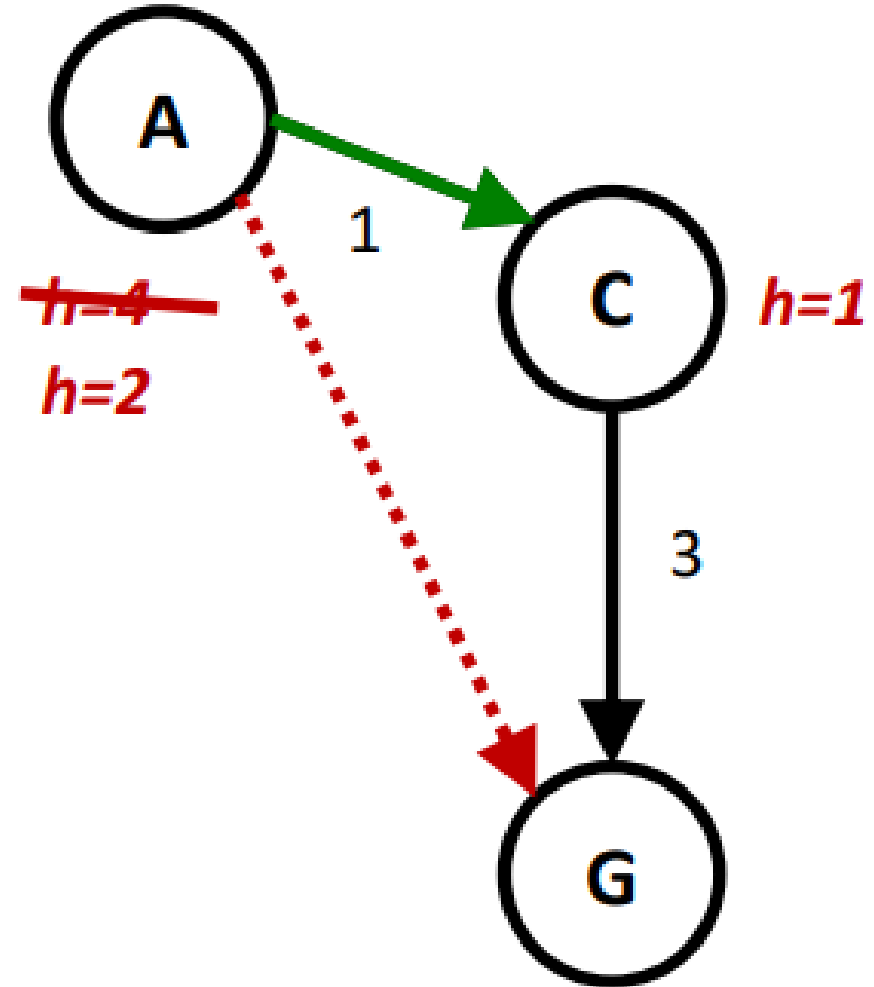
Búsqueda en grafos

- Nunca expandir un estado dos veces
- Implementación
 - Búsqueda en árbol + conjunto de estados que han sido expandidos
 - Búsqueda en árbol normal pero antes de expandir un nodo revisar que no ha sido expandido antes
- ¡Necesitamos consistencia!



Consistencia en heurísticas

- El costo estimado de una heurística $\leq$ que el costo real
 - Admisible
 - Costo de heurística $\leq$ costo real a la meta
 - $h(v) \leq h^*(v) \forall v \in V$
 - Consistencia
 - Costo heurístico de un arco $\leq$ costo de cada arco
 - $h(u) - h(v) \leq d(u, v) \forall (u, v) \in E$
- Consecuencia
 - $h(A) \leq \text{cost}(A \text{ to } C) + h(C)$
 - ¡A* es óptimo!



Consistencia: Distancia en línea recta

Condición de consistencia: $h(n) \leq c(n, a, n') + h(n')$ para todo sucesor n' .

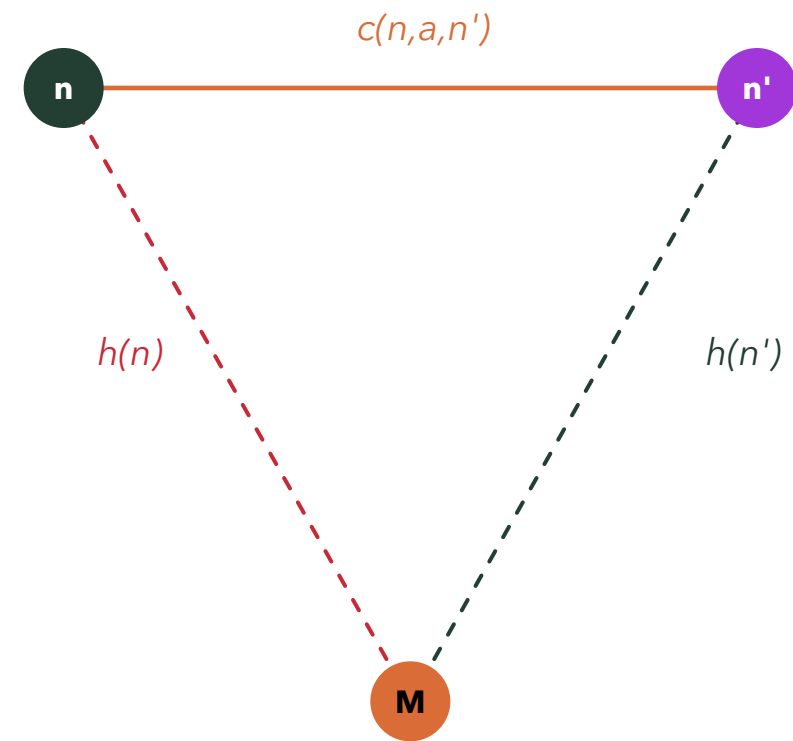
Con h_SLD , esto equivale a la desigualdad del triángulo:

- $d(n, meta) \leq d(n, n') + d(n', meta)$,
- donde $d(\cdot, \cdot)$ es la distancia euclidiana.

Prueba: la desigualdad del triángulo es una propiedad fundamental de la distancia euclidiana.

- El lado directo $n \rightarrow meta$ es siempre $\leq$ que el camino $n \rightarrow n' \rightarrow meta$.
- Por lo tanto $h_SLD(n) \leq c(n, a, n') + h_SLD(n')$ para cualquier acción a . ■

Consecuencia: consistencia $\Rightarrow$ admisibilidad, y A^* sobre grafos es óptimo.



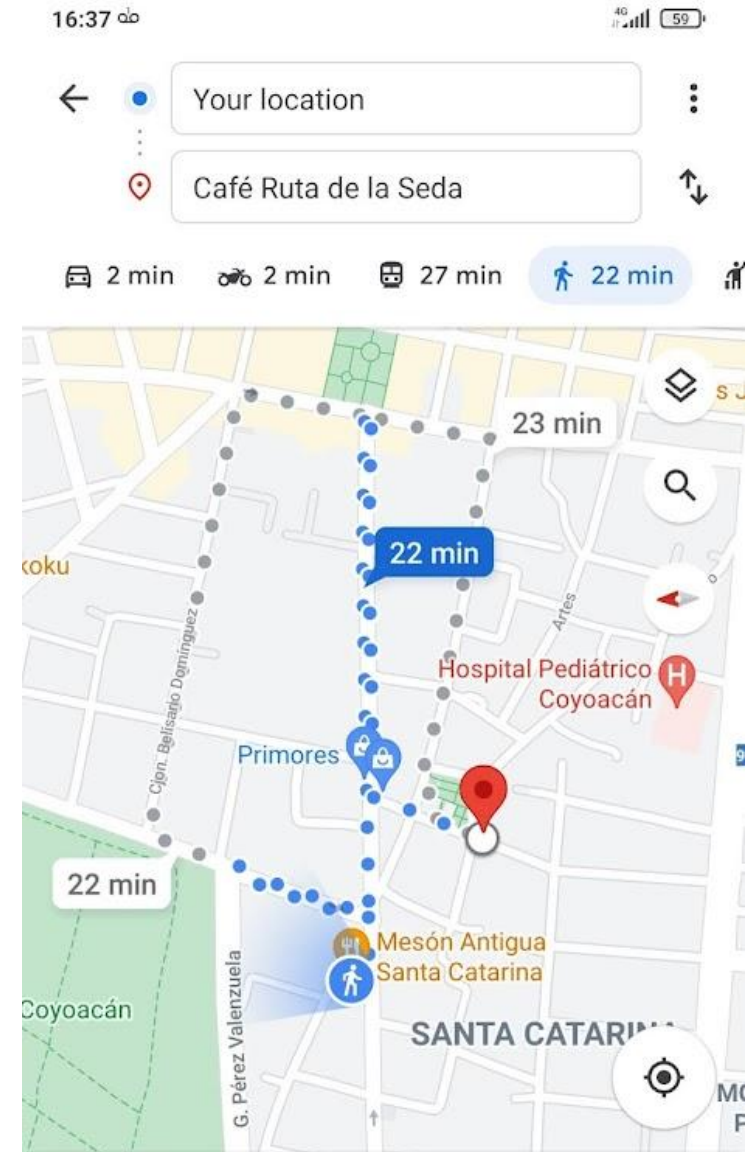
$$h(n) \leq c(n, a, n') + h(n')$$

El algoritmo

```
function GRAPH-SEARCH(problem, fringe) return a solution, or failure
  closed ← an empty set
  fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node ← REMOVE-FRONT(fringe)
    if GOAL-TEST(problem, STATE[node]) then return node
    if STATE[node] is not in closed then
      add STATE[node] to closed
      for child-node in EXPAND(STATE[node], problem) do
        fringe ← INSERT(child-node, fringe)
      end
    end
  end
```

Algunos comentarios

- No olviden quitar nodos que ya visitaron (usar un conjunto)
- El agente no prueba todos los planes en el mundo real
- Planear es acerca de simular
- La búsqueda es tan buena como lo sea el modelo
- Y los errores pasan...



Ahora relajémonos y resolvamos un sudoku...

- ¿Búsqueda en anchura?
- ¿Búsqueda en profundidad?
- ¿A*?

	2			8	9		4	7
		9	7		4			
	6	4				9		8
	5	3	9	7	8			6
		6		4		7	9	
		7		3		2		5
6		1	5	9		8	2	4
5		2	8		6	3		9
	3	8			7	5	6	1

Para la otra vez...

- Problemas de satisfacción de restricciones

The End.